

Note: Spelling Bee Game - C# Implementation

Panoramica Generale

I tre file C# implementano il gioco "Spelling Bee" in versioni progressive, da una versione base a versioni più ottimizzate con LINQ e strutture dati avanzate.

Program (3).cs - Versione Base

Scopo

Implementazione del gioco Spelling Bee che:

- Carica una lista di parole da file
- Filtra le parole secondo regole specifiche
- Ordina le parole per punteggio
- Calcola il punteggio totale

Dati di Input

```
Lettera centrale obbligatoria: 'U' (seed[0])  
Lettere disponibili: { 'U', 'X', 'L', 'T', 'A', 'E', 'N' }  
File: "C:/Academy/A04/valid_words_spelling_bee.txt"
```

Regole del Gioco

1. **Lunghezza minima:** 4 lettere
2. **Lettera obbligatoria:** Deve contenere 'U' (primo elemento del seed)
3. **Lettere ammesse:** Solo le lettere presenti in seed
4. **Pangram:** Parola che utilizza TUTTE le 7 lettere del seed (bonus punteggio)

Calcolo del Punteggio

- **Parole 4 lettere:** 1 punto
- **Parole > 4 lettere non-pangram:** lunghezza della parola
- **Parole pangram:** lunghezza + 7 punti bonus
- Le parole pangram vengono visualizzate in **verde**

Strutture Dati Utilizzate

Array vs List

```
// Array: dimensione fissa, non modificabile
var words = File.ReadAllLines(...); // O(1) accesso, non posso aggiun-
gere/rimuovere

// List: dimensione variabile
List<string> valid = new();           // O(1) accesso, Add() e Remove()
disponibili
```

Sequenze di Accesso Casuale (Random Access)

- **Array**: accesso $O(1)$, dimensione fissa
- **List**: accesso $O(1)$, dimensione variabile

Sequenze Non-Random Access

- **Stack**: LIFO (Last-In-First-Out), Push() per inserire, Pop() per estrarre
- **Queue**: FIFO (First-In-First-Out), Enqueue() per inserire, Dequeue() per estrarre

Complessità Computazionale (Big O)

```
Ordinamento con bubble sort:  $O(n^2/2)$   $\approx O(n^2)$ 
Accesso ad array/list:  $O(1)$ 
Iterazione su collezione:  $O(n)$ 
Ricerca in array:  $O(n)$ 
```

Funzioni Principali

IsValid(word) - $O(n \cdot m)$

Verifica se una parola è valida:

- Controlla lunghezza e 4
- Verifica presenza della lettera obbligatoria (seed[0])
- Verifica che tutte le lettere siano nel seed

```
bool IsValid(string word) {
    if (word.Length < 4) return false;
    bool found = false;
    foreach (var ch in word)
```

```

    if (ch == seed[0]) { found = true; break; }
    if (!found) return false;
    foreach (var ch in word) {
        found = false;
        foreach (var ch2 in seed)
            if (ch == ch2) { found = true; break; }
        if (!found) return false;
    }
    return true;
}

```

IsPangram(word) - $O(n \cdot m)$

Verifica se la parola usa tutte le 7 lettere del seed

```

bool IsPangram(string word) {
    foreach (var ch in seed) {
        bool found = false;
        foreach (var ch2 in word)
            if (ch2 == ch) { found = true; break; }
        if (!found) return false;
    }
    return true;
}

```

GetScore(word) - $O(n \cdot m)$

Calcola il punteggio della parola

```

int GetScore(string word) {
    if (word.Length <= 4) return 1;
    else if (IsPangram(word)) return word.Length + 7;
    else return word.Length;
}

```

InOrder(a, b) - Comparatore

Determina l'ordine tra due parole (punteggio decrescente, alfabetico crescente)

```

bool InOrder(string a, string b) {
    int sa = GetScore(a), sb = GetScore(b);
    if (sa != sb) return sa > sb;
}

```

```
    return a.CompareTo(b) < 0;
}
```

Algoritmo di Ordinamento

Bubble sort con ottimizzazione (j parte da i+1):

```
for (int i = 0; i < valid.Count - 1; i++) {
    for (int j = 1 + i; j < valid.Count; j++) {
        if (!InOrder(valid[i], valid[j])) {
            string tmp = valid[i];
            valid[i] = valid[j];
            valid[j] = tmp;
        }
    }
}
```

Program (4).cs - Versione Ottimizzata con LINQ

Novità Principali

1. Uso di LINQ (Language Integrated Query)

```
// Versione ottimizzata con catena LINQ
foreach (var data in words.Where(IsValid)
    .Select(GetScore)
    .OrderByDescending(a => a.Score)
    .ThenByDescending(a => a.Word))
```

2. Operatori Logici Abbreviati

```
// Utilizzo di && con cortocircuito (short-circuit evaluation)
bool IsValid(string word)
=> word.Length >= 4
&& word.Contains(seed[0])
&& word.All(seed.Contains);

// Nota: l'ordine è importante per efficienza - condizioni più restrittive
prima
```

3. Tuple per Ritornare Più Valori

```
(string Word, int Score, bool Pangram) GetScore(string word) {  
    bool pangram = seed.All(word.Contains);  
    int score = (word.Length > 4 ? word.Length : 1) + (pangram ? 7 : 0);  
    return (word, score, pangram);  
}  
  
// Utilizzo  
var a = (Word: "Hello", Score: 3, Pangram: true);
```

4. Analisi Frequenza Lettere

```
Dictionary<char, int> freq = new();  
foreach (var ch in File.ReadAllText(...))  
    if (ch is >= 'A' and <= 'Z') {  
        if (freq.ContainsKey(ch))  
            freq[ch]++;  
        else  
            freq[ch] = 1;  
    }  
  
// Pattern matching: ch is >= 'A' and <= 'Z'
```

5. Metodi LINQ Utili

```
bool allOdd = aaa.All(x => x % 2 == 1);    // Tutti gli elementi soddisfano  
la condizione  
bool anyOdd = aaa.Any(x => x % 2 == 1);    // Almeno un elemento soddisfa  
la condizione  
var unique = words.Distinct();             // Rimuove duplicati  
var top7 = freq.OrderByDescending(x => x.Value).Take(7); // Top 7  
elementi
```

6. Misura Prestazioni

```
Stopwatch sw = Stopwatch.StartNew();  
// ... codice da misurare ...  
sw.Stop();  
Console.WriteLine($"Tempo: {sw.Elapsed.TotalMilliseconds} ms");
```

Miglioramenti rispetto a Program (3)

- **Codice più conciso:** LINQ riduce il codice iterativo
- **Efficienza:** Operatori && cortocircuito riducono computazioni inutili
- **Leggibilità:** Catene LINQ esprimono il flusso logico chiaramente
- **Prestazioni:** Monitoraggio tramite Stopwatch

Program (5).cs - Metodi Object e GetHashCode

Oggetti Fondamentali di Ogni Classe

1. ToString()

```
double f = Math.PI;  
var z = f.ToString(); // Converte in stringa  
Console.WriteLine(z);
```

Uso: Conversione di qualsiasi oggetto in rappresentazione testuale

2. Equals(object obj)

```
double f = Math.PI;  
double f2 = 3.14;  
if (f.Equals(f2))  
    Console.WriteLine("Uguali");  
else  
    Console.WriteLine("Diversi");
```

Uso: Confronto tra oggetti (personalizzabile in classi custom)

3. GetHashCode()

```
foreach (var w in File.ReadLines(...))
    Console.WriteLine($"{w, -12} has this hashCode : {((uint)w.GetHashCode()) % 37}");
```

Uso: Genera un identificatore numerico (int32) unico per un oggetto

- Usato internamente da hashtable, dictionary, hashset
- `% 37` limita il range dell'hashCode

HashSet - Rimozione Duplicati

Concetto

Struttura dati che non consente duplicati, basata su hashCode

Utilizzo

```
int[] a = { 3, 5, 18, 9, 5, 17, 3, 2, 100, 18 };

// Modo 1: LINQ Distinct()
foreach (var s in a.Distinct())
    Console.Write($"{s}"); // Stampa: 3 5 18 9 17 2 100

// Modo 2: HashSet con Add() che ritorna bool
HashSet<int> set = new HashSet<int>();
foreach (var n in a) {
    if (set.Add(n)) // Add restituisce true se aggiunto (non era presente)
        Console.Write($"{n}");
}
```

Proprietà di HashSet

- **Add(item):** Aggiunge elemento se non presente, restituisce bool
- **Remove(item):** Rimuove elemento, restituisce bool indicante successo
- **Contains(item):** Verifica presenza elemento
- **Count:** Numero di elementi
- **Complexity:** $O(1)$ medio per Add/Remove/Contains

Distinzione tra Collezioni

Collezione	Accesso	Duplicati	Ordinamento
Array	Random $O(1)$	Sì	No

List	Random O(1)	Sì	No
HashSet	No	No	No
Dictionary<K,V>	K'V O(1)	No (chiavi)	No
Queue	FIFO	Sì	FIFO
Stack	LIFO	Sì	LIFO

Note Importanti sulla Lettura File

Path con Slash vs Backslash

```
// Corretto: usa / per compatibilità cross-platform
File.ReadAllLines("C:/Academy/A04/valid_words_spelling_bee.txt");

// Problematico: \ può causare conflitti con escape sequences (\n, \t)
File.ReadAllLines("C:\\Academy\\A04\\valid_words_spelling_bee.txt");
```

Differenze Array e List

```
// Array
var words = File.ReadAllLines(...); // .Length per dimensione
// List
List<string> valid = new();           // .Count per numero elementi
```

Concetti Avanzati

Pattern Matching

```
// Pattern matching con range
if (ch is >= 'A' and <= 'Z') { ... }

// Tuple deconstruction
(var x, var y) = ptA;
(valid[i], valid[j]) = (valid[j], valid[i]); // Swap
```


Lambda Expressions

```
// Sintassi
x => x % 2 == 1           // Un parametro
(x, y) => x + y           // Più parametri
x => { return x * 2; }    // Body con statement
```

Expression-bodied Members

```
// Tradizionale
bool IsValid(string word) { return ...; }

// Expression-bodied
bool IsValid(string word) => word.Length >= 4 && word.Contains(seed[0])
&& word.All(seed.Contains);
```

Ordine di Esecuzione - Program (3)

1. Caricamento parole da file in array
2. Iterazione su tutte le parole
3. Filtraggio parole valide (IsValid) ' List
4. Ordinamento lista con bubble sort ($O(n^2)$)
5. Per ogni parola valida:
 - Calcolo punteggio (GetScore)
 - Stampa con colore (verde se pangram)
 - Accumulo punteggio totale
6. Stampa punteggio totale

Riassunto Comparativo

Aspetto	Program 3	Program 4	Program 5
Approccio	Imperativo classico	LINQ funzionale	Métodi Object
Ordinamento	Bubble sort manuale	OrderBy/ThenBy LINQ	N/A
Leggibilità	Media	Alta	Media

Prestazioni	Buone	Ottime	Varia
Comunità	Didattico	Moderno	Fondamentale